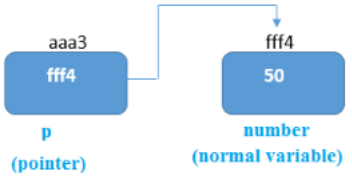
Pointers

C – Pointers

We can get the **memory address** of a variable with the reference operator &:

Eg:



int Age = 43; // an int variable
printf("%d", Age); // Outputs the value of Age (43)
printf("%p", &Age); // Outputs the memory address of Age (0x7ffe5367e044)

- A **pointer** is a variable that **stores** the **memory address** of another variable as its value.
- A **pointer variable points** to a **data type** (like int) of the same type, and is created with the * operator.
- The address of the variable you are working with is assigned to the pointer:
- C pointer is the derived data type that is used to store the address of another variable and can also be used to access and manipulate the variable's data stored at that location.

C – Pointers

Syntax:
type *var-name;

Example

```
int Age = 43;      // An int variable
int* ptr = &Age;   // A pointer variable, with the name ptr, that stores the address of Age
printf("%d\n", Age); // Output the value of Age (43)
printf("%p\n", &Age); // Output the memory address of Age (0x7ffe5367e044)
printf("%p\n", ptr); // Output the memory address of Age with the pointer (0x7ffe5367e044)
```

Explanation:

- Create a pointer variable with the name ptr, that **points to** an int variable (Age).
- Note that the type of the pointer has to match the type of the variable you're working with (int in our example).
- Use the & operator to store the memory address of the Age variable, and assign it to the pointer.
- Now, ptr holds the value of Age's memory address.

C – Initializing a pointer

Pointer Initialization

- After declaring a pointer variable, you need to initialize it with the address of another variable using the **address of (&) operator**. This process is known as **referencing a pointer**.

Syntax

pointer_variable = &variable;

Example

int x = 10;

int *ptr = &x;

Here, **x** is an integer variable, **ptr** is an integer pointer. The pointer **ptr** is being initialized with **x**.

C – Dereference operator

- In the example above, we used the pointer variable to get the memory address of a variable (used together with the & **reference** operator).
- You can also get the value of the variable the pointer points to, by using the ***** operator (the **Dereference** operator):

Example

- int Age = 45; // Variable declaration
  int* ptr = &Age; // Pointer declaration

  printf("%p\n", ptr); // Reference: Output the memory address of Age with the pointer (0x7ffe5367e044)

  printf("%d\n", *ptr); // Dereference: Output the value of Age with the pointer (45)

Note that the * sign can be confusing here, as it does two different things in our code:
•When used in declaration (int* ptr), it creates a **pointer variable**.
•When not used in declaration, it act as a **dereference operator**.

C – Importance of pointers

- Pointers are one of the things that make C stand out from other programming languages, like Python and Java.
- With pointers, you can access and modify the data located in the memory, pass the data efficiently between the functions, and create dynamic data structures like linked lists, trees, and graphs.
- They are important in C, because they allow us to manipulate the data in the computer's memory. This can reduce the code and improve the performance. And sometimes you even have to use pointers, for example when working with files and memory management.
- **But be careful**; pointers must be handled with care, since it is possible to damage data stored in other memory addresses.

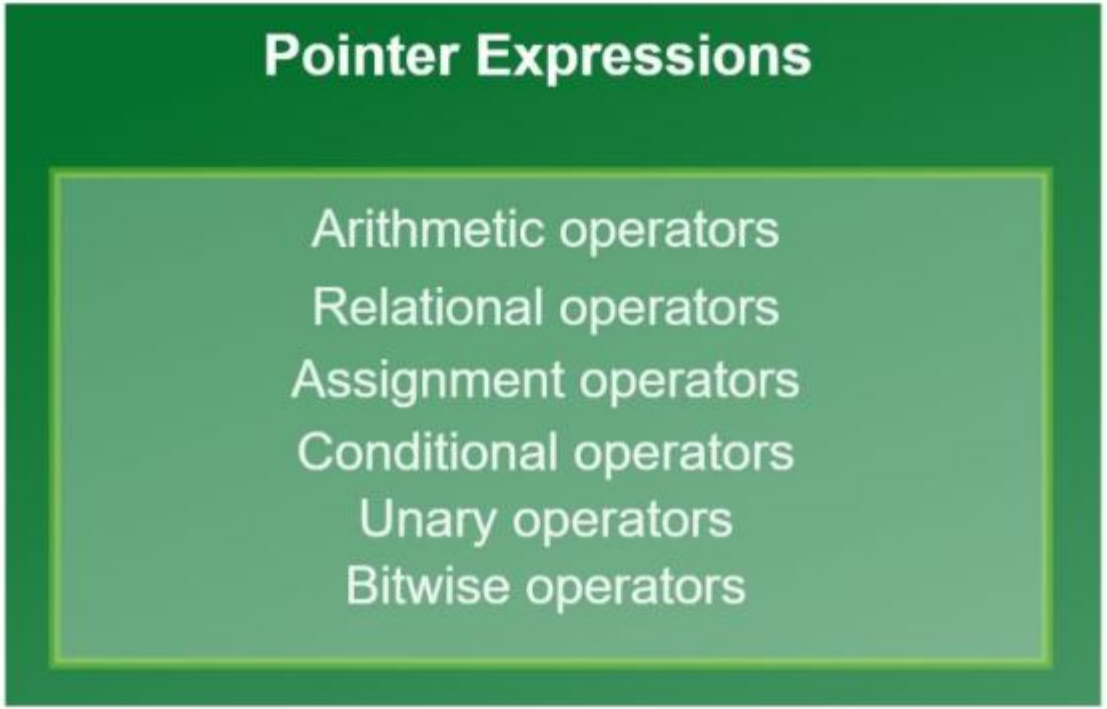
C – Access and Manipulate Values using Pointer

- The value of the variable which is pointed by a pointer can be accessed and manipulated by using the pointer variable.
- You need to use the asterisk (*) sign with the pointer variable to access and manipulate the variable's value.

```
#include <stdio.h>
int main()
{
   int x = 10;
   int * ptr = & x; // Pointer declaration and initialization
   printf("Value of x = %d\n", * ptr); // Printing the current value
   ptr = 20; // Changing the value
   printf("Value of x = %d\n", * ptr);  // Printing the updated value
   return 0;
}
```

Pointer Expressions

Just like any other variable, these operations can be also **performed on pointer variables**.



Arithmetic operations using pointers

- We can add an integer or subtract an integer using a pointer pointing to that integer variable.
- Operations possible (Addition, subtraction, multiplication, division & modulo)

Examples:

*ptr1 + *ptr2
*ptr1 * *ptr2
*ptr1 + *ptr2 - *ptr3

While performing division, make sure you put a blank space between '/' and '*' of the pointer as together it would make a multi-line comment('/*').

- Can also directly perform arithmetic expressions on integers by dereferencing pointers.

***p1 + 10, *p2 - 5, *p1 - *p2 + 10, *p1/2**

Similarly we can write expressions using relational, conditonal, unary, assignment, bitwise operators also.

Additional problems - https://cse.poriyaan.in/topic/pointer-expressions-and-pointer-arithmetic--50374/

Pointers and arrays

- You can also use pointers to access arrays.

Example

```
int myNumbers[4] = {25, 50, 75, 100};
int i;
for (i = 0; i < 4; i++) {
  printf("%d\t", myNumbers[i]);   //25    50     75      100
}
for (i = 0; i < 4; i++) {
    printf("%p\n", &myNumbers[i]);  //0x7ffe70f9d8f0   0x7ffe70f9d8f4
                                                0x7ffe70f9d8f8   0x7ffe70f9d8fc
}
```

Note that the last number of each of the elements' memory address is different, with an addition of 4.

Pointers and arrays

How Are Pointers Related to Arrays

The **name of an array**, is actually a **pointer** to the **first element** of the array.

```
int myNumbers[4] = {25, 50, 75, 100};
printf("%p\n", myNumbers);          // 0x7ffe70f9d8f0
printf("%p\n", &myNumbers[0] ;   // 0x7ffe70f9d8f0
printf("%d", *myNumbers);     //25
```

Since myNumbers is a pointer to the first element in myNumbers, you can use the * operator to access it:

To access the rest of the elements in myNumbers, you can increment the pointer/array (+1, +2, etc):

```
printf("%d\n", *(myNumbers + 1));   // 50
printf("%d", *(myNumbers + 2));      // 75
```

Pointers and arrays

How Are Pointers Related to Arrays

We can also use **loops** to access the array elements

```
int myNumbers[4] = {25, 50, 75, 100};
int *ptr = myNumbers;
int i;

for (i = 0; i < 4; i++)
 {
  printf("%d\t", *(ptr + i));     //25    50    75     100
}
```

We can update the value of array elements with pointers:

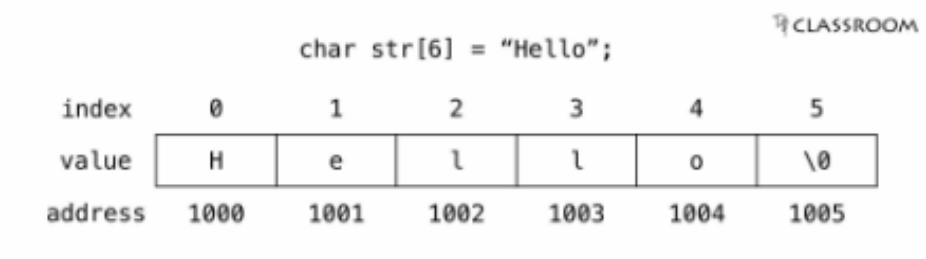
```
*myNumbers = 13; //1st element
*(myNumbers +1) = 17; //2nd
```

Pointers and Strings

- We are creating a string str using char character array of size 6.

  char str[6] = "Hello";

  The above string can be represented in memory as follows.



- Each character in the string str takes 1 byte of memory space.

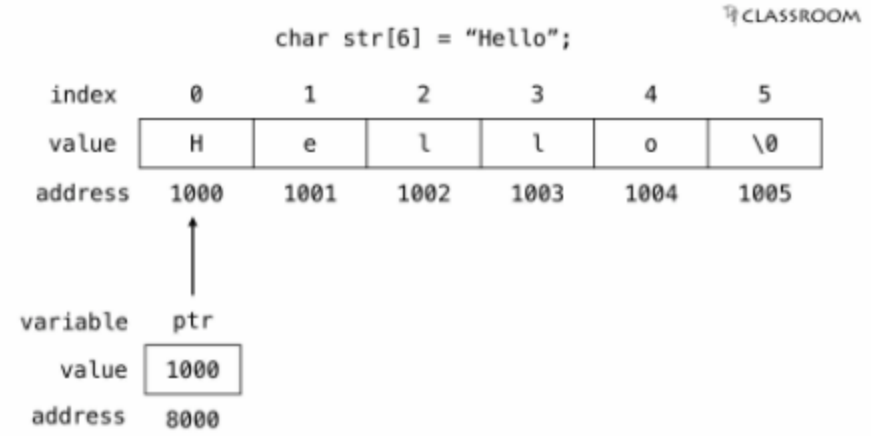
Pointers and Strings

Creating a pointer for the string

- The variable name of the string str holds the address of the first element of the array i.e., it points at the starting memory address.
- So, we can create a character pointer ptr and store the address of the string str variable in it. This way, ptr will point at the string str.

char *ptr = str;

The pointer variable ptr is allocated memory address 8000 and it holds the address of the string variable str i.e., 1000.



Pointers and Strings

Accessing string via pointer

To access and print the elements of the string we can use a loop and check for the \0 null character.

```
#include <stdio.h>
int main()
{
   char str[6] = "Hello";      // string variable
   char *ptr = str;            // pointer variable
   while(*ptr != '\0')         // print the string
   {
        printf("%c", *ptr);
         ptr++;                // move the ptr pointer to the next memory location
   }
   return 0;
}
```

Array of strings using pointer

We can create a two dimensional array and save multiple strings in it.

char city[4][12] = { "Chennai", "Kolkata", "Mumbai", "New Delhi" };

- The problem with this approach is that we are allocating 4x12 = 48 bytes memory to the city array and we are only using 33 bytes.

```
char city[4][12] = {
  "Chennai",
  "Kolkata",
  "Mumbai",
  "New Delhi"
};
```

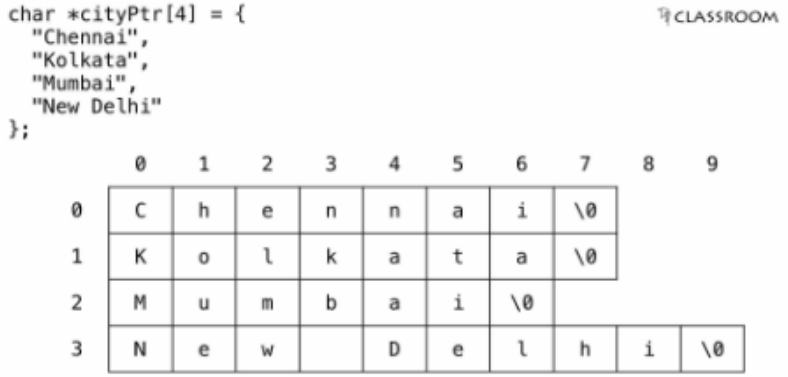
CLASSROOM

	0	1	2	3	4	5	6	7	8	9	10	11
0	C	h	e	n	n	a	i	\0				
1	K	o	l	k	a	t	a	\0				
2	M	u	m	b	a	i	\0					
3	N	e	w		D	e	l	h	i	\0		

Array of strings using pointer

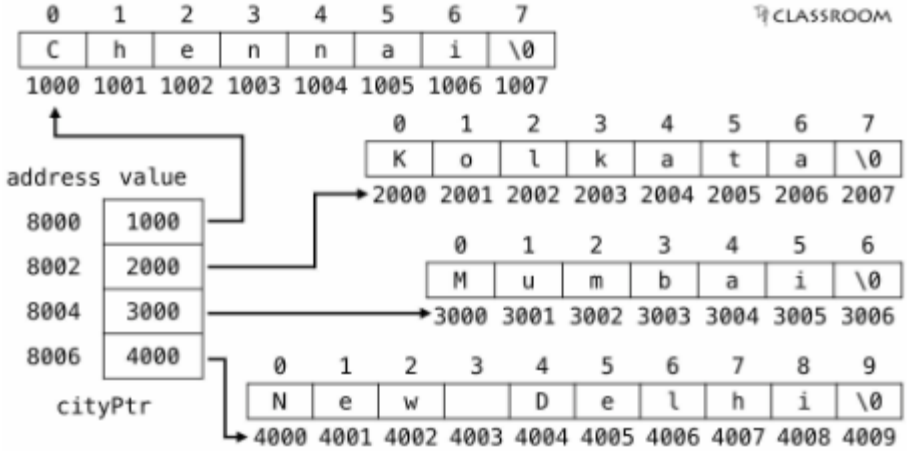
- We can save those unused memory spaces by using pointers as shown below.

```
char *cityPtr[4] = { "Chennai",
                     "Kolkata",
                     "Mumbai",
                     "New Delhi" };
```



The above array of pointers can be represented in memory as follows.

- The cityPtr pointer variable is allocated the memory address 8000 to 8007. Assuming integer address value takes 2 bytes space. So, each pointer gets 2 bytes.
- Name of the cities are saved in locations 1000, 2000, 3000 and 4000.



Array of strings using pointer

```
#include <stdio.h>
 int main()
 { // array of pointers
       char *cityPtr[4] = { "Chennai", "Kolkata", "Mumbai", "New Delhi" };
       // temporary variable
      int r, c; // print cities
       for (r = 0; r < 4; r++)
           {
                c = 0;
                while(*(cityPtr[r] + c) != '\0')
               {
                    printf("%c", *(cityPtr[r] + c));
                    c++;
               }
               printf("\n");
           }
         return 0;
}
```

In the code we are using the r variable to access each row of the pointer. And we are using the c variable to access each character in a selected row.

```
Chennai
Kolkata
Mumbai
New Delhi
```

Array of Pointers in C

- In C, a pointer array is a homogeneous collection of indexed pointer variables that are references to a memory location.
- It is generally used in C Programming when we want to point at multiple memory locations of a similar data type in our C program.
- We can access the data by dereferencing the pointer pointing to it.

Syntax:

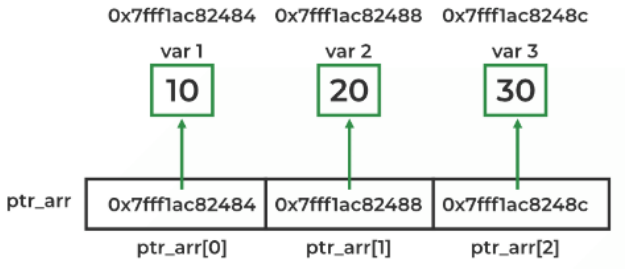
pointer_type *array_name [array_size];

Here,

- **pointer_type:** Type of data the pointer is pointing to.
- **array_name:** Name of the array of pointers.
- **array_size:** Size of the array of pointers.

Array of Pointers in C

```
// C program to demonstrate the use of array of pointers
 #include <stdio.h>
int main()
{
int var1 = 10; int var2 = 20; int var3 = 30;
int* ptr_arr[3] = { &var1, &var2, &var3 }; // array of pointers to integers
for (int i = 0; i < 3; i++)                    // traversing using loop
{
  printf("Value of var%d: %d\tAddress: %p\n", i + 1, *ptr_arr[i], ptr_arr[i]);
}
return 0;
}
```




```
Value of var1: 10 Address: 0x7fff1ac82484
Value of var2: 20 Address: 0x7fff1ac82488
Value of var3: 30 Address: 0x7fff1ac8248c
```

As shown in the above example, each element of the array is a pointer pointing to an integer. We can access the value of these integers by first selecting the array element and then dereferencing it to get the value.